

DEVELOPMENT_HISTORY

📊 Development History & Progress

Milestone Timeline

Phase 1 — Core Backend Setup

- Set up Python 3.13 development environment in VS Code.
- Built the initial FastAPI application (`main.py`) and ran it locally with Uvicorn (`py -m uvicorn main:app --reload`).
- Defined the in-memory hospital data structure (`data.py`) covering beds, doctors, nurses, equipment, and patients.

Phase 2 — API Layer

- Implemented `GET /status` to expose current hospital data.
- Added `POST /update` with Pydantic models (e.g. `HospitalUpdate`) to validate incoming data from the frontend.
- Enabled CORS (`CORSMiddleware`) so the frontend (served separately, e.g. via Live Server) could call the backend without cross-origin errors.

Phase 3 — Frontend Dashboard

- Built `index.html` with sections for hospital overview, bed occupancy, doctors, nurses, equipment, patients, and alerts.
- Styled the dashboard with CSS (cards, progress bars, status badges, responsive layout).
- Wired up JavaScript to read form inputs, build JSON, and call the backend via `fetch()`.

Phase 4 — Optimization Logic

- Implemented availability and occupancy calculations ($\text{Available} = \text{Total} - \text{Occupied}$, $\text{Occupancy \%} = \text{Occupied} / \text{Total} \times 100$).
- Added automatic status classification (`NORMAL` / `WARNING` / `CRITICAL`).
- Extended the same logic to doctors, nurses, and equipment (busy/in-use calculations).

Phase 5 — Recommendation Engine

- Added `GET /recommendations`, applying rule-based thresholds (bed shortages, doctor shortages, nurse shortages, ventilator shortages) to generate alert messages.

Phase 6 — Visualization

- Integrated `Chart.js` to visualize resource utilization by department.
- Connected chart rendering to live backend data via `loadResourceChart()`.

Phase 7 — Simulation Features

- Added `POST /simulate/surge` to model an emergency surge (increased occupancy, reduced staff/equipment availability, increased patients).
- Added `POST /simulate/normal` to reset the hospital state to baseline values for repeatable demos.

Phase 8 — Access Control (Prototype)

- Added `login.html` and a `/login` endpoint with hardcoded demo credentials to demonstrate the concept of role-based access, explicitly scoped as non-production.

Current State

A working end-to-end prototype: input → JSON → FastAPI → calculation → rule-based alerts → live dashboard update, including two simulation modes for demoing hospital behavior under normal and surge conditions.

Known Limitations

- **In-memory data only** — restarting the server resets all values back to `data.py` defaults; no persistent database yet.
- **Rule-based, not ML-based** — recommendations use fixed thresholds rather than learned models.
- **Prototype-level login** — hardcoded credentials, not secure authentication (no hashing, sessions, or token-based auth).

Roadmap / Next Steps

1. **Persistent storage** — replace the in-memory dictionary with a database (e.g. PostgreSQL/SQLite) so data survives restarts and historical trends can be tracked.
2. **Real authentication** — hashed passwords, session/JWT-based login, and role-based permissions (admin vs. staff).
3. **ML-based forecasting** — use historical admission/occupancy data to predict surges and recommend proactive resource allocation instead of purely reactive thresholds.
4. **Multi-hospital / multi-ward support** — extend the data model beyond a single facility.
5. **Notifications** — push/email/SMS alerts when a resource crosses into CRITICAL status.
6. **Automated tests** — unit tests for the calculation and recommendation logic, and integration tests for the API endpoints.